

Sleep VS GPA in College: Predicción del GPA en base al sueño en estudiantes de carrera.

Alfredo Alejandro Soto Herrera - A01711368

Abstract – En el presente documento se implementa un modelo de Machine Learning con el fin de lograr predecir el GPA de estudiantes universitarios en base a las horas de sueño que diariamente tienen, además de su calidad, etc. Esto con el fin de encontrar una correlación entre ambas situaciones, más no una causalidad. Se utilizó un modelo de regresión lineal multivariable junto con una función de activación ReLU, MSE para calcular el costo o pérdida y Batch Back Propagation para corregir todos los pesos y biases.

Keywords – IA, Machine learning, Dataset, ETL, EDA, Regresión lineal, ReLU, MSE, Bias.

I. INTRODUCCIÓN

La inteligencia artificial es un concepto que ha rondado en la humanidad desde hace muchos años, se cree que es algo reciente pero sus inicios se pueden remontar desde el siglo XVIII, esto debido a que en 1726 la novela titulada como “Los viajes de Gulliver”, escrita por Jonathan Swift habla de un concepto nuevo en donde se introduce a una “máquina” que ayuda a los eruditos a escribir y a tener nuevas ideas [1]. Desde entonces podemos estar hablando de algo no humano lo cual ayuda a la sociedad.

A pesar de tener ideas de IA, el concepto no se aterrizó hasta que en 1950 Alan Turing publica “Computing Machinery and Intelligence” en donde se dejan estas ideas fugaces que pueden interpretarse hoy en día como conceptos relacionados a IA, sino que todo se concentra en este concepto. Turing plantea si en algún punto las máquinas podrán exhibir un comportamiento inteligente sin el apoyo humano [1]. Además de crear la prueba de Turing para poder medir la inteligencia.

Turing sentó las bases e hizo que muchos científicos se pusieran a investigar en esta área, ya que le veían potencial, pero durante casi 30 años de investigación, no se logró a los resultados esperados, se creó el perceptrón, lo que conocemos como una neurona en Deep learning. Pero no fue hasta que IBM logró vencer al campeón de ajedrez en 1997 [2]. Esto despertó al mundo después de esos 30 años de avances sin resultados, entonces en 2012 AlexNet vuelve a alborotar el avispero [3], ganó la competencia ImageNet con un amplio margen con respecto al segundo lugar y así sucedían cosas que

convencían a la comunidad científica que la Inteligencia Artificial tenía un camino claro, Machine Learning.

Machine Learning no es otra cosa que un área dentro de la Inteligencia Artificial, con los casos de éxito anteriores, es que se ha seguido el camino de ML (abreviatura de Machine Learning) a pesar de existir otras áreas. Lo que hace ML a grandes rasgos es que puede aprovechar un gran número de datos y aplicar con ellos una serie de algoritmos para poder generar un modelo que pueda detectar patrones en los datos y de esta forma poder predecir nuevas relaciones en datos desconocidos [4].

El objetivo de este paper es trabajar dentro del área de Machine Learning para poder crear un modelo y hacer predicciones con él dentro de un dataset. El dataset con el que se estará trabajando fue extraído de Kaggle [5], titulado como “Sleep vs. GPA in College”, en este dataset, se analiza el promedio de estudiantes universitarios contrastándolo con sus horas y calidad de sueño.

Analizar este caso es fundamental ya que la presión que sufren los estudiantes por obtener buenas calificaciones puede llegar a derivar en problemas de depresión o incluso de ideación suicida, en la UNAM se mostró que cerca del 39.6% de los estudiantes tienen ideas suicidas [6]. Está claro que no es la única causa o la causa principal de esto, pero mientras se puedan reducir las posibilidades, este trabajo tiene un valor.

Se utilizará un modelo de regresión lineal para poder entrenar un modelo que nos permita predecir el promedio estudiantil en base a su sueño.

II. EDA Y ETL

A. EDA inicial

EDA, significa “Análisis Exploratorio de los Datos”, nos permite conocer con lo que estamos trabajando y poder encontrar formas de interpretar los datos de manera visual o numérica. En este caso se hará un EDA inicial para conocer los datos y así poder realizar un ETL con certeza de lo que se está haciendo en él, finalmente se realiza un segundo EDA para corroborar que los datos se encuentren limpios y cómo los deseamos.

El dataset elegido [6], cuenta con 21 features y 1 variable a predecir y 634 instancias.

Nombre	Descripción
student_id	ID anónimo por estudiante, es único.
study	Identificador de universidad.
university	Universidad a la que el estudiante atiende.
semester	Número de semestre en el que se hizo la medición.
cohort_code	Abreviaturas para identificar a las universidades.
gender	Género reportado por la misma universidad.
first_generation	Indica 1 si ninguno de sus padres completó o inició la universidad y 0 si alguno de los 2 si la terminó.
underrepresented	Indica 1 si alguno de sus padres pertenece a alguna minoría (negro, latino, hispano, nativo americano, de alguna isla del pacífico). 0 si no aplica.
avg_sleep_minutes	Promedio en minutos de lo que duerme el estudiante, sin contar el tiempo recostado antes de dormir.
avg_sleep_hours	Promedio del sueño, pero en este caso en horas y no minutos.
daytime_sleep_minutes	Es el tiempo que se duerme en el día medido en minutos.
sleep_midpoint_minutes	La hora promedio que queda a la mitad del periodo de sueño, expresada en minutos desde medianoche.
bedtime_variability	Qué tanto varía la hora a la que el estudiante se va a dormir entre diferentes noches.
nights_tracked_fraction	Proporción de noches en las que Fitbit registró datos de sueño válidos durante el periodo de estudio.
prior_gpa	GPA promedio del periodo anterior.
term_gpa	GPA obtenido por el estudiante en este periodo.
gpa_change	Resta entre el prior_gpa y term_gpa para ver la razón de cambio entre ellos.
term_units	Cantidad de créditos académicos cursados en el periodo.
term_load_z	Indica qué tan por encima o por debajo del promedio está la carga académica.
under_6h_sleep	Marca con 1 si el estudiante duerme menos de 6 horas al día, con 0 si tiene 6 o más horas.

sleep_bracket	Divide las horas que duerme el estudiante en rangos.
sleep_midpoint_clock	Lo mismo que sleep_midpoint_minutes, pero expresado como hora legible en lugar de minutos.

Tabla 1. Variables en el Dataset.

Conociendo las variables y su función nos podemos dar una idea de las columnas que vamos a eliminar porque no nos van a servir para predecir “term_gpa”, pero antes de ello vamos a analizar un poco más los datos.

En este EDA inicial pudimos revisar los features que contaban con valores nulos, si se repetían instancias, etc. Todo con el fin de poder descubrir como vienen los datos y que features no necesitamos.

Valores Faltantes	Valores faltantes	Porcentaje
gender	3	0.473186
first_generation	4	0.630915
underrepresented	1	0.157729
term_units	147	23.186120
term_load_z	147	23.186120

Figura 1. Features con valores faltantes.

En la Figura 1 podemos ver que hay 5 features con valores faltantes, pero en 3 de ellas no hay problema por que son pocos valores, en cambio en “term_units” y en “term_load_z” falta un 23% de los datos, lo cual si es una gran cantidad como para simplemente completar con un promedio. En el sitio de Kaggle en donde se encuentra el dataset [5], se nos avisa que faltaban esos datos porque pertenecen a una universidad que no los dio.

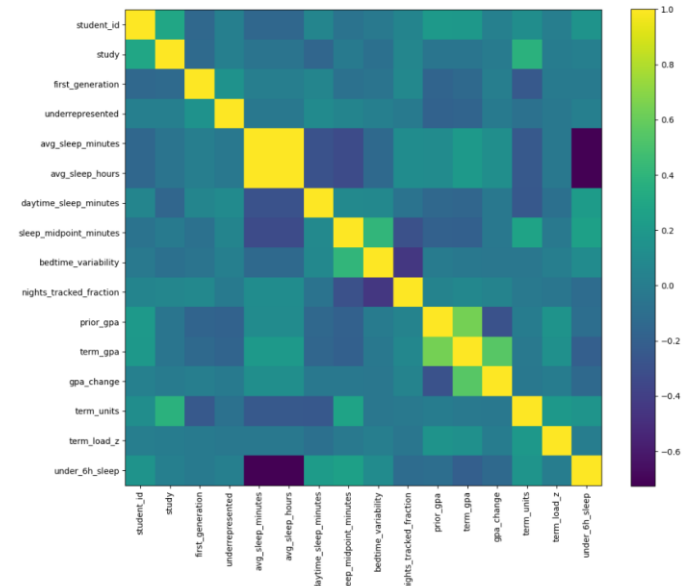


Figura 2. Matriz de Correlación

De igual forma se hizo una matriz de correlación para poder observar que features no sirven para poder predecir la variable

“term_gpa”, en donde se pudo encontrar que no hay algún feature que tenga una relación clara con “term_gpa”, con lo cual dependemos de más de una variable para poderla predecir.

```
Correlación con GPA
term_gpa      1.000000
prior_gpa     0.638025
gpa_change    0.548917
avg_sleep_minutes 0.201674
avg_sleep_hours 0.201542
student_id    0.196675
term_load_z   0.140652
nights_tracked_fraction 0.074180
term_units   -0.030672
bedtime_variability -0.035967
study        -0.056451
first_generation -0.135061
daytime_sleep_minutes -0.153059
underrepresented -0.170013
sleep_midpoint_minutes -0.194537
under_6h_sleep -0.199581
Name: term_gpa, dtype: float64
```

Figura 3. Valores de Correlación con GPA.

Hay que aclarar que los features que más relación tienen son “prior_gpa” y “gpa_change” las cuales también muestran el GPA, con lo cual surge la posibilidad de eliminarlas ya que nos pueden generar data leakage.

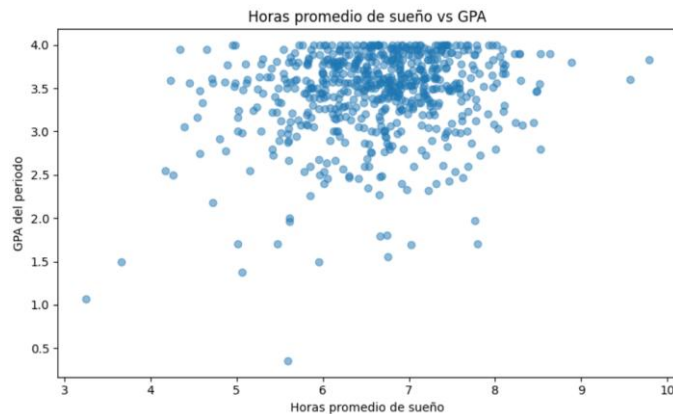


Figura 4. Scatterplot de las Horas de Sueño contra GPA.

Finalmente se realizó un Scatterplot para poder ver si llega a haber una relación entre las horas de sueño y el GPA, en donde podemos notar que la mayor parte de los datos se encuentra entre 6 y 7 horas de sueño de 3.0 GPA para arriba, podemos observar casos en donde a pesar de dormir 8 horas no logró un alto GPA, o con pocas horas de sueño si tiene un GPA muy alto.

B. ETL

Para comenzar con el proceso de transformación en el ETL, lo primero fue borrar aquellas features que nos dimos cuenta de que no necesitábamos.

Columna Eliminada	Razón
student_id	Es un identificador para los estudiantes, no nos ayuda a predecir.

study	Es otro identificador para las universidades.
university	No necesitamos el nombre de la universidad para predecir.
semester	Saber el semestre en el que está el estudiante tampoco ayuda para poder predecir.
cohort_code	Otro identificador de universidades que no se necesita.
avg_sleep_minutes	Es lo mismo que “avg_sleep_hours”, entonces es redundante.
gpa_change	Al derivar de “term_gpa” ya que es la resta de este con el anterior GPA, puede ocasionar data leakage.
term_units	Tiene 147 valores faltantes y se decidió no hacer imputación.
term_load_z	Igual que el caso anterior, tiene muchos datos faltantes.
under_6h_sleep	La información que nos da es redundante ya que la podemos obtener directamente de “avg_sleep_hours”.
sleep_bracket	Igual que el caso anterior, es redundante a “avg_sleep_hours”.
sleep_midpoint_clock	Es la representación de “sleep_midpoint_minutes” pero en formato de horario, por lo que nos quedamos con la variable numérica.

Tabla 2. Features Eliminadas

Como se observó en la Figura 1, teníamos 5 variables con valores faltantes, después de eliminar features nos quedamos solamente con 3, en los 3 casos realizamos una imputación utilizando la moda, ya que las 3 variables se tratan casos binarios, hombre o mujer en “género”, 1 o 0 en “underrepresented” y de igual forma 0 y 1 en “first_generation”, por lo que al usar la moda simplemente se agarra el valor con mayor frecuencia. En este último se detectó un caso inválido, ya que hubo una sola respuesta con valor de 2, lo cual no es válido, entonces se transformó a NA para hacer la imputación junto con los otros 4 valores NA.

	variable	Q1	Q3	IQR	limite_inferior	\
0	first_generation	0.00000	0.0000	0.00000	0.000000	
1	underrepresented	0.00000	0.0000	0.00000	0.000000	
2	avg_sleep_hours	6.11250	7.1700	1.05750	4.526250	
3	daytime_sleep_minutes	23.10000	51.2500	28.15000	-19.125000	
4	sleep_midpoint_minutes	345.17500	437.6250	92.45000	206.500000	
5	bedtime_variability	0.07470	0.2917	0.21700	-0.250800	
6	nights_tracked_fraction	0.82100	1.0000	0.17900	0.552500	
7	prior_gpa	3.23225	3.7895	0.55725	2.396375	
8	term_gpa	3.23300	3.8100	0.57700	2.367500	
	limite_superior	cantidad_outliers	porcentaje_outliers			
0	0.000000	103	16.246057			
1	0.000000	119	18.769716			
2	8.756250	11	1.735016			
3	93.475000	29	4.574132			
4	576.300000	15	2.365931			
5	0.617200	94	14.826498			
6	1.268500	54	8.517350			
7	4.625375	17	2.681388			
8	4.675500	21	3.312303			

Figura 5. Análisis en busca de Outliers.

Después de la imputación, hay que buscar outliers en los datos, por ello es que calculando el primer y tercer cuartil es que logramos identificar el número de outliers como se observa en la Figura 5, ahora depende de nosotros ver que outliers reportados en verdad lo son y no solo valores atípicos pero posibles por la naturaleza de la variable. Por ejemplo, “avg_sleep_hours” tiene 11 outliers marcados, pero con un máximo de 9.7 horas, lo cual es algo más que posible y un inferior de 4.5 horas lo cual sí es poco, pero aun así es algo que pasa por lo cual no hay outliers en esta variable.

Las variables que sí son un poco sospechosas son “daytime_sleep_minutes”, “sleep_midpoint_minutes” y “bedtime_variability”. Con lo cual, rescatamos una serie de boxplots hechos en el proceso de EDA Inicial.

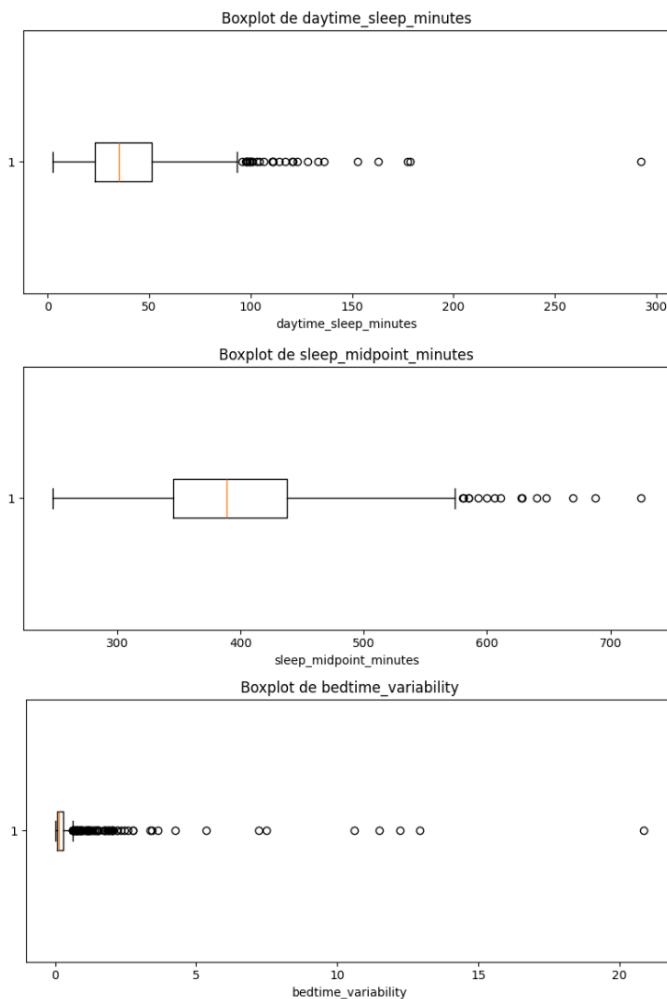


Figura 6. Box Plots de variables sospechosas.

En la Figura 6, podemos darnos cuenta de los valores que sobresalen en estas 3 variables. Primero “daytime_sleep_minutes” tiene un máximo de 297 minutos lo cual es mucho para dormir durante el día, además de ser el único valor con esta magnitud, por lo cual se va a eliminar la instancia con ese valor.

Después en la variable “sleep_midpoint_minutes” podemos

ver como también llega a haber outliers, pero en este caso, no vamos a eliminar ni cambiar nada ya que hay varios valores cerca y no lo vamos a catalogar como un error o algo extraño.

Finalmente, en el boxplot de “bedtime_variability” encontramos algunos outliers a partir de un valor de 5, al ser muy pocos valores, se van a considerar como outliers, por lo que se borrarán aquellas instancias con un valor de 5 o mayor en esta variable.

Con esto daríamos por terminado el proceso de Transformación y pasaríamos a corroborar que los datos están como queremos.

C. EDA Final

Se tomó la decisión de tener un EDA Inicial y uno Final con el objetivo de en un inicio observar cómo estaban los datos y en este caso, el final es para poder corroborar lo hecho en el ETL y poder decidir si pasar todas las variables a nuestro modelo.

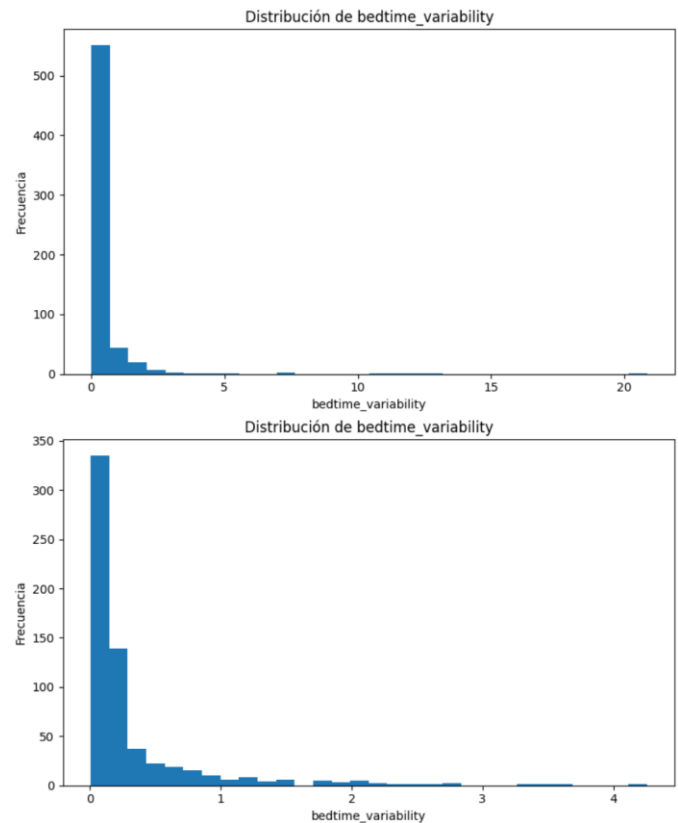


Figura 7. Comparación de Distribuciones

Se pueden apreciar 2 gráficas en la Figura 7, la de arriba pertenece al EDA Inicial, antes de hacer el ETL y la de abajo es ya después del ETL, en donde podemos notar que al borrar los outliers mejora la distribución de los datos.

Este fue el primer paso para corroborar lo hecho y contrastarlo con el inicio, también se volvió a obtener una matriz de correlación, pero es mejor verla desde los datos.

```

Correlación con term_gpa
term_gpa      1.000000
prior_gpa     0.640222
avg_sleep_hours 0.176082
nights_tracked_fraction 0.100847
daytime_sleep_minutes -0.099399
first_generation -0.138393
underrepresented -0.176749
bedtime_variability -0.214564
sleep_midpoint_minutes -0.219471
Name: term_gpa, dtype: float64

```

Figura 8. Datos de la matriz de correlación.

Con estos datos contrastados con la variable a predecir “term_gpa” podemos ver que hay algunas variables que no están muy relacionadas, esto no indica que las vamos a descartar automáticamente del modelo, pero es algo a tener en cuenta, más cuando se termine el primer modelo y se intente mejorar.

Finalmente obtuvimos correlación entre las variables independientes para buscar si hay alguna que se “repita” o sea redundante con otra, se obtuvo que no había una correlación mayor de 0.7 entre las variables, el cual fue nuestro colchón para activar o no la alarma.

III. MODELO

Al enfrentarnos con un problema de regresión, el modelo a seguir en este proyecto tiene de base la Regresión Lineal Multivariable.

A. Preprocesamiento

Antes de pasar a la implementación del modelo, después de haber terminado con el EDA y ETL, extraemos el nuevo dataset ya listo para su uso en el modelo, es lo que diría si no fuera importante el preprocesamiento, ya que en el logramos hacer lo siguiente,

- Transformamos la variable categórica “gender” a 0’s y 1’s.
- Separamos el dataset en X y Y, siendo X la matriz de las variables independientes. Y siendo la variable para predecir “term_gpa”.
- Separamos las X en 60% Train, 20% Test y 20% Validation.
- Finalmente estandarizamos todos los valores para que no haya variables que dominen a otras o que el gradiente se mueva de manera desigual.

B. Parámetros e Hiperparámetros

Empezando por los parámetros, a los pesos se decidió inicializarlos bajo el método HE, el cual nos arroja la fórmula:

```
W1 = np.random.randn(n_features, n_hidden) * np.sqrt(2.0 / n_features)
```

Figura 9. Inicialización con HE

Lo que nos dice esta fórmula es que iniciemos con un valor aleatorio para después multiplicarlo por la raíz cuadrada de 2 entre el número de features. Lo que esto nos permite es que, en el primer recorrido de la red, las neuronas no mueran al tener una activación de 0, así nos aseguramos de que al inicio todo funcione bien.

Para los Hiperparámetros se decidió tener una capa oculta con 16 neuronas, learning rate de 0.05 y 2000 epochs. Se decidieron los valores de manera arbitraria y al probarlo funcionó bien.

C. Función de Activación ReLU

Se tomó la decisión de tener una función de activación, primero para poder agregarle complejidad al modelo y no solamente quedarnos con la capa en la que se ingresan las X y directamente pasa a la salida, sino tener también una capa oculta para que el modelo pueda dejar de ser simplemente lineal.

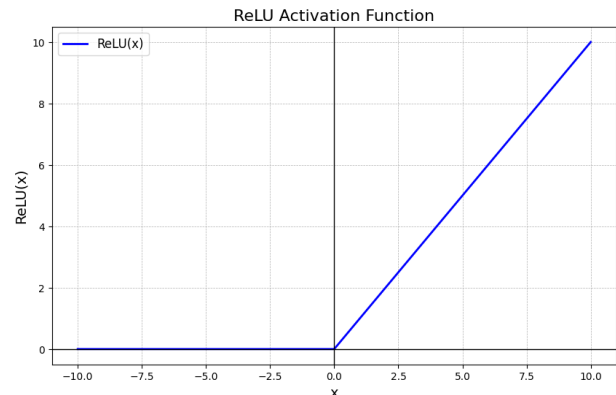


Figura 10. Función ReLU

Teniendo en cuenta lo anterior, se escogió ReLU como función de activación debido a su gran eficiencia computacional, además de que a lo largo de los años ha demostrado mejorar tareas en el aprendizaje profundo.

D. Regresión Lineal Multivariable

Llegamos al punto de explicar lo importante de este modelo, ya que, a pesar de escoger la función de costo, activación, por una razón, digamos que siguen siendo decisiones comunes y no se alejan a lo que realmente se hace con normalidad. En cambio, escoger una función de hipótesis si que es algo más único, ya que cada problema es diferente. En esta ocasión, partiremos de una regresión lineal multivariable, ya que nos permite resolver problemas de regresión como es el caso, es simple computacionalmente hablando y multivariable porque contamos con muchas X, las cuales nos permitirán predecir la Y con mayor precisión en lugar de enfocarnos solo en una X.

E. Función de Costo MSE

Después de que estuvimos recorriendo cada neurona y calculando predicciones con la ayuda de nuestra función de hipótesis, llegamos a la capa de salida en donde necesitamos calcular la función de pérdida o costo para poder saber que tan alejados estamos de un valor ideal.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Figura 11. Fórmula de MSE (Mean Square Error)

Para ello, nuestra función de costo será MSE (Mean Square Error), utilizamos MSE en lugar de MAE ya que nos permite distinguir entre valores positivos y negativos, cosa que MAE nos la complica más, ya que al arrojar un valor absoluto no sabremos que era antes del absoluto. La elección se cerró a estas 2 opciones porque ambas nos ayudan en problemas de regresión, a diferencia por ejemplo de Cross Entropy la cual es para problemas de clasificación más que para un valor continuo.

F. Back Propagation

Una vez que recorrimos toda la red al inicio, tenemos que dar vuelta atrás para poder saber en dónde tenemos que ajustar los pesos y los biases y así obtener la menor pérdida que podamos lograr, esto gracias a los epochs que son las vueltas que le damos a todo el dataset en este caso, ya que, al separar en Train, Test y Validation quedamos con un Train algo pequeño por lo que no hay mini-batches, así que se da una vuelta completa a todos los datos de entrada.

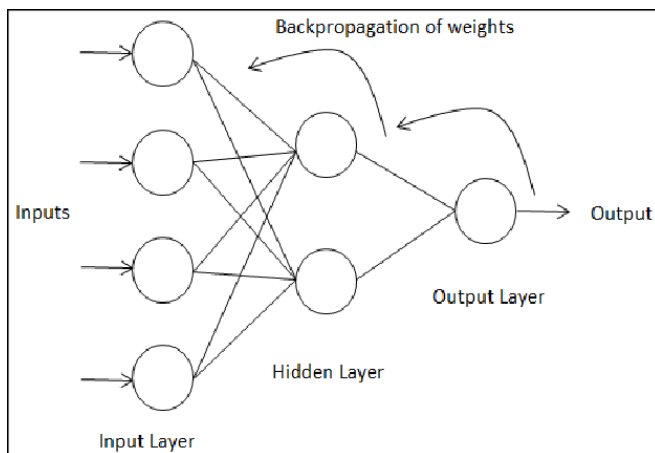


Figura 12. Back Propagation

En resumen, Back Propagation es un paso imposible de dejar de lado si se quiere llegar a un modelo que logre predecir con cercanía a los valores reales. Back Propagation va de la mano con el siguiente punto del modelo.

G. Batch Gradient Descent

Gradient Descent no es otra cosa que una función de optimización que junto a back Propagation permite mejorar el modelo, por medio del cambio de los pesos sinápticos y los biases.

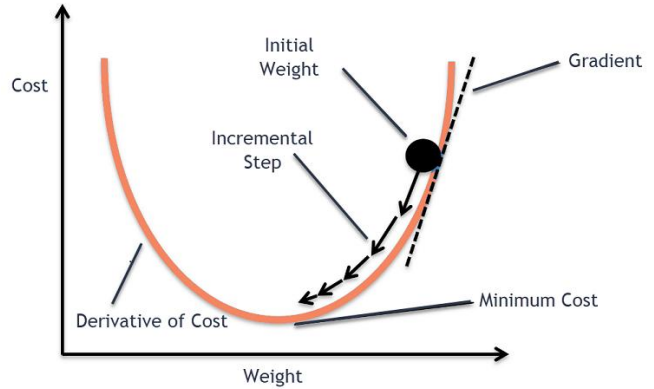


Figura 13. Gradient Descent

La forma en que logra optimizar nuestros parámetros es por medio del gradiente, es decir, utiliza las derivadas parciales de las neuronas y sus salidas Z y A hasta llegar al peso o bias que se va a ajustar.

Como se mencionó en el punto pasado, va de la mano con back Propagation para ir encontrando los gradientes desde el output hasta el inicio y así recorrer cada punto del modelo y calcular el cambio de los parámetros.

H. Resultados

Una vez explicado todas las decisiones que se tomaron para crear el modelo es que llegamos a los resultados, es decir, que descubrimos o a qué llegamos luego de aplicar todo lo mencionado en este artículo hasta este punto.

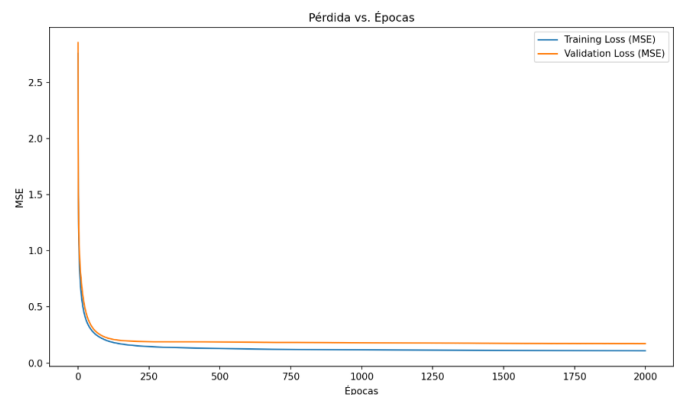


Figura 14. Train vs Validation

Primero observamos esta gráfica en busca de Overfitting, la cual parece no haber a pesar de que a partir del 250 epoch se empiezan a separar, pero parece ser un valor mínimo, insignificante para poder saber si se llega a presentar

Overfitting, con lo cual quedaría descartada esta opción al observar la gráfica.

```
MSE en prueba: 0.1480
RMSE en prueba: 0.3847
MAE en prueba: 0.2808
R^2 en prueba: 0.2193
```

Figura 15. Valores para medir el rendimiento del modelo

Por otra parte, en la consola imprimimos los valores que se muestran en la Figura 15 ya que nos permiten ver cómo se comporta nuestro modelo en realidad.

MSE y MAE nos muestran el error que tenemos al terminar con todos los epochs, si no tenemos alguna métrica con qué comparándolo es difícil comprenderlo. Por ello RMSE es un mejor valor para darnos una idea del funcionamiento, nos muestra el error medido en la misma unidad que el GPA, es decir, si un alumno tiene un GPA de 3.5, el modelo puede dar un valor de ± 0.38 , un intervalo de (3.12, 3.88). Lo cual, al tener una escala de GPA de 0 a 4, puede ser un intervalo grande de error a pesar de ser decimal.

Finalmente, la R^2 nos dice qué proporción de variabilidad del GPA explica el modelo, al obtener un valor de 0.21 podemos decir que las variables no tienen una gran representación del GPA, lo que al mismo tiempo es realista ya que el sueño no necesariamente afecta las calificaciones de un estudiante.

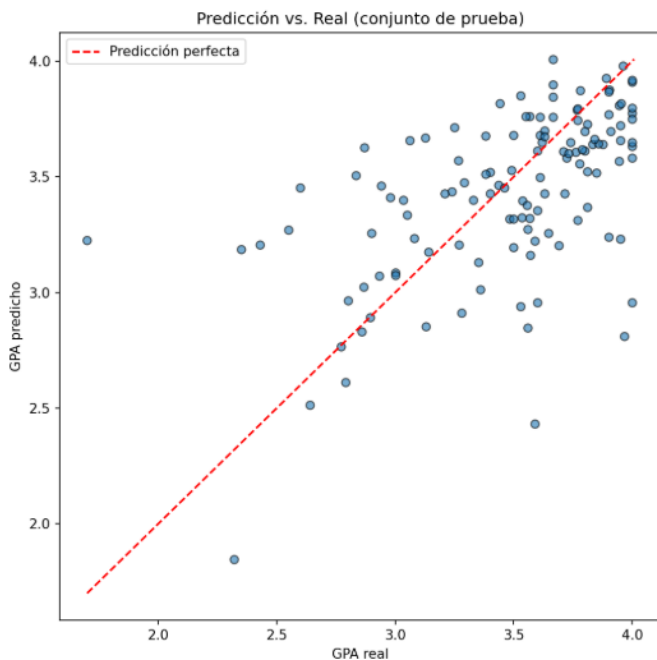


Figura 16. Predicción vs Real

La última gráfica o output que se generó en el código es la mostrada en la Figura 16, en ella podemos ver la comparación de los valores reales en el eje x, contra los valores predichos por nuestro modelo, en ella podemos notar que hay muchos

valores que están lejos de su valor real, otros valores que se acercan y otros pocos que, sí que llegan a tener el valor correcto, esto nos dice que hay una gran dispersión, muchos de los puntos están lejos de la línea roja que presenta el valor perfecto o deseado al predecir los reales.

I. Observaciones

Continuando con lo visto en los resultados es necesario analizar lo que se logra ver.

- El modelo sobreestima a los estudiantes con GPA bajo (menor a 3.0), es decir, estudiantes con GPA real de 1.7 pudieron obtener un valor predicho de 3.2.
- Las predicciones caen en un rango de 2.8 a 4.0, cuando los valores reales están de 1.7 a 4.0, esto se complementa con el punto anterior.
- Pasa lo mismo con los estudiantes de alto rendimiento, ya que los sigue sobreestimando, va de rangos de 3.5 a 4.0 cuando en la realidad están en 2.4 a 4.0.
- Si se llegara a usar este modelo para poderle seguir el paso a un estudiante que está teniendo GPA's bajos, le estimaría un valor mucho más alto de lo que probablemente vaya a conseguir.

IV. CONCLUSIÓN

Este paper presenta solo un avance de todo el proyecto, es pronto para poder concluir algo al respecto.

APÉNDICE

Repositorio del proyecto:

<https://github.com/AlejandroSH1/Implementaci-n-de-Aprendizaje-M-quina>

REFERENCIAS

- [1] Mucci, T. La historia de la IA. IBM. (2024, 21 octubre). <https://www.ibm.com/es-es/think/topics/history-of-artificial-intelligence>
- [2] S.A. Deep Blue. IBM. (s.f.). <https://www.ibm.com/history/deep-blue>
- [3] S.A. AlexNet — El primer gran éxito de una CNN que cambió el reconocimiento de imágenes. Zeromathai. (s.f.) <https://zeromathai.com/es/alexnet-es/>
- [4] S.A. ¿Qué es el Machine Learning? AWS Amazon. (s.f.). <https://aws.amazon.com/es/what-is/machine-learning/>
- [5] Mind Lab. Sleep vs. GPA in College. Kaggle. (s.f.). https://www.kaggle.com/datasets/kylefengkfeng209/sleep-vs-gpa-in-college?select=college_sleep_and_gpa.csv
- [6] Cuamba N, Flores G, González E. Depresión e Ideación Suicida en Estudiantes Universitarios: Un estudio Exploratorio con Perspectiva Género. UNAM, (2024). <https://ru.iiec.unam.mx/6634/>